



Облачные платформы и виртуализация

Задание 2. Работа с Docker.

Познакомимся поближе с Docker.

Используйте машину, подготовленную с помощью Vagrant.

Этап 1. Работа с контейнером.

Осуществим поиск по всем образам, содержащим ubuntu в их названии.

```
docker search ubuntu
```

Первый результат в списке имеет самый большой рейтинг и помечается как официальный образ.

Для загрузки на машину данного образа:

```
docker image pull ubuntu
```

Команда эквивалента:

```
docker image pull ubuntu:latest
```

В результате выполнения любой из двух вышеуказанных команд у нас будет локально загружена последняя версия контейнера ubuntu.

Если нужен конкретный контейнер, то укажите версию:

```
docker image pull ubuntu:18.04
```

Список доступных локальных образов можно узнать так:

```
docker image ls
```

Запустим контейнер, с определенной версией контейнера:

```
docker container run -it ubuntu:18.04
```

Оказавшись в шелле контейнера, мы можем выполнить команды внутри контейнера:

```
ls
```

```
ps ax
```

К примеру, установим ПО:

```
apt-get update
```

```
apt-get install inetutils-ping
```

Теперь попробуем пингануть сайт:

```
ping ya.ru
```

Чтобы выйти из контейнера не завершая его, нажмите Ctrl+P затем Ctrl+Q

Глянем список работающих контейнеров

```
docker container ls
```



Облачные платформы и виртуализация

Результат будет похож на пример:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
35ac9218a880	ubuntu:18.04	"/bin/bash"	8 minutes ago	Up 8 minutes		cocky_fermat

Вернемся к контейнеру, для этого обратите внимание на его id (замените пример на свой):

docker container attach 35ac9218a880

Альтернативно можем использовать имя вместо id, в нашем примере - cocky_fermat

Для выхода из контейнера:

exit

Глянем на список еще раз:

docker container ls

И заметим, что контейнера нет! Так как мы завершили процесс в контейнере.

Чтобы увидеть весь список контейнеров, включая те, которые не работают:

docker container ls -a

Создадим контейнер без его старта:

docker container create -it --name ubuntu-20 ubuntu /bin/bash

Далее стартанем его:

docker container start -ai ubuntu-20

Выйдем из контейнера не завершая его, нажмите Ctrl+P затем Ctrl+Q

Далее остановим его:

docker container stop ubuntu-20

Давайте создадим контейнеры, запустим их в фоновом режиме. Используем образ Alpine (небольшой, стандартный образ):

docker container run -d --name dummy1 alpine sleep 1d

Для еще одного контейнера используем образ Busybox (еще меньше Alpine):

docker container run -d --name dummy2 busybox sleep 1d

Глянем список образов для сравнения

docker image ls

Видите, на сколько Alpine и Busybox меньше?

Глянем на список контейнеров еще раз:

docker container ls

Представьте, что было много запущенных контейнеров, и нам нужен способ остановить их все



Облачные платформы и виртуализация

Нам понадобится способ извлечь только их идентификаторы контейнеров, например в виде списка и передать его команде `stop`

Список всех запущенных контейнеров можно извлечь с помощью:

```
docker container ls -q
```

Теперь, чтобы объединить это с командой `stop`, мы можем выполнить следующее

```
docker container stop $(docker container ls -q)
```

А как насчет остановленных контейнеров? Можем ли мы как-то их удалить? Да, мы можем

Давайте запросим их список с помощью

```
docker container ls -a
```

Остановленный контейнер или список контейнеров, которые мы можем удалить передайте команде:

```
docker container rm ubuntu-20
```

Все остановленные контейнеры можно удалить

```
docker container prune
```

После удаления теперь видно что список пуст:

```
docker container ls -a
```

Проверим список образов в системе:

```
docker image ls
```

Мы можем удалить образ

```
docker image rm ubuntu
```

Проверим список образов в системе:

```
docker image ls
```

Образ удалился

Этап 2. Работа с другим реестром.

Итак, мы знаем, что Docker Hub — это реестр по умолчанию

Но как мы можем изменить его, хотя бы временно, только для одного контейнера?

Просто передайте реестр во время команды `run`:

```
docker container run -d --name other -p 8080:80 <адрес реестра>
```